

# **LAPORAN PRAKTIKUM STRUKTUR DATA**

## **Implementasi algoritma sorting pada java: shell sort, quick sort, merge sort dan bubble sort GUI**



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**  
**PADANG**

**2026**

## KATA PENGANTAR

Puji dan syukur kita panjatkan kehadiran Allah yang Maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil praktikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada Bapak Wahyudi selaku dosen dalam mata kuliah praktikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan praktikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang praktikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajian laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran-saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2026

Shofwatul Hasanah Sarma

# DAFTAR ISI

|                                       |                                     |
|---------------------------------------|-------------------------------------|
| KATA PENGANTAR .....                  | i                                   |
| DAFTAR ISI.....                       | ii                                  |
| BAB I.....                            | 3                                   |
| PENDAHULUAN .....                     | <b>Error! Bookmark not defined.</b> |
| 1.1    Latar Belakang.....            | 3                                   |
| 1.2    Tujuan Praktikum.....          | 3                                   |
| 1.3    Manfaat Praktikum.....         | 3                                   |
| BAB II.....                           | 4                                   |
| PEMBAHASAN.....                       | 4                                   |
| 2.1    Pengertian .....               | 4                                   |
| 2.2    Praktikum .....                | 4                                   |
| BAB III .....                         | 85                                  |
| KESIMPULAN.....                       | 85                                  |
| 3.1    Ringkasan Hasil Praktikum..... | 85                                  |
| 3.2    Saran Pengembangan .....       | 85                                  |
| DAFTAR PUSTAKA .....                  | 86                                  |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Pengolahan data dalam pemrograman sering membutuhkan proses pengurutan agar data lebih mudah dibaca, dicari, dan dianalisis. Algoritma sorting seperti Shell Sort, Quick Sort, Merge Sort, dan Bubble Sort digunakan untuk menyusun data dari nilai terkecil ke terbesar atau sebaliknya. Pada materi ini, pengurutan tidak hanya ditulis dalam bentuk program console, tetapi juga divisualisasikan melalui GUI agar proses sorting lebih mudah dipahami. Shell Sort memanfaatkan konsep gap, Quick Sort menggunakan pivot dengan teknik median of three, Merge Sort membagi data menjadi dua bagian lalu menggabungkannya kembali, sedangkan Bu.

### 1.2 Tujuan Praktikum

Tujuan praktikum ini adalah memahami cara kerja beberapa algoritma sorting pada Java, membandingkan proses pengurutan antar algoritma, dan melatih kemampuan mengimplementasikan sorting dalam bentuk program console maupun GUI. Praktikum ini juga bertujuan agar mahasiswa dapat melihat perbedaan karakteristik setiap algoritma, seperti penggunaan gap pada Shell Sort, pivot pada Quick Sort, rekursi pada Merge Sort, serta visualisasi langkah pada Bubble Sort GUI.

### 1.3 Manfaat Praktikum

Manfaat praktikum ini adalah menambah pemahaman tentang struktur data dan algoritma sorting, meningkatkan kemampuan logika pemrograman, serta melatih mahasiswa membaca alur rekursi, perbandingan data, dan pertukaran elemen. Selain itu, visualisasi GUI membantu proses belajar menjadi lebih interaktif dan mudah dipahami.

# BAB II

## PEMBAHASAN

### 2.1 Pengertian

Sorting adalah proses mengurutkan data agar tersusun secara sistematis.

- 1 Shell Sort adalah pengembangan dari Insertion Sort yang membandingkan elemen berdasarkan jarak tertentu atau gap.
- 2 Quick Sort adalah algoritma pengurutan berbasis pivot; pada materi ini pivot dipilih dengan teknik Median of Three agar lebih efisien.
- 3 Merge Sort adalah algoritma divide and conquer yang membagi array menjadi bagian kecil lalu menggabungkannya kembali dalam keadaan terurut.
- 4 Bubble Sort GUI adalah tampilan Bubble Sort yang memperlihatkan proses pertukaran elemen secara bertahap melalui antarmuka grafis.

### 2.2 Praktikum

#### 1. Kelas shell sort java

- ⇒ Program ini digunakan untuk mengurutkan data menggunakan algoritma shell sort. Shell sort merupakan pengembangan dari insertion sort yang bekerja dengan membandingkan elemen elemen yang memiliki jarak tertentu (gap) sehingga proses pengurutan menjadi lebih cepat dibandingkan insertion sort biasa.

#### A. Kode program

```
package pekan8_2511533015;

public class ShellSort_2511533015 {

    public static void ShellSort_2511533015(int[] A_3015) {
        int n_3015 = A_3015.length;
        int gap_3015 = n_3015 / 2;

        while (gap_3015 > 0) {
```

```

        for (int i_3015 = gap_3015; i_3015 < n_3015; i_3015++) {
            int temp_3015 = A_3015[i_3015];
            int j_3015 = i_3015;

            while (j_3015 >= gap_3015 && A_3015[j_3015 - gap_3015] > temp_3015) {
                A_3015[j_3015] = A_3015[j_3015 - gap_3015];
                j_3015 = j_3015 - gap_3015;
            }

            A_3015[j_3015] = temp_3015;
        }
        gap_3015 = gap_3015 / 2;
    }
}

public static void main(String[] args_3015) {

    int[] data_3015 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

    System.out.print("Sebelum: ");
    printArray_3015(data_3015);

    ShellSort_2511533015(data_3015);

    System.out.print("Sesudah (Shell Sort): ");
    printArray_3015(data_3015);
}

public static void printArray_3015(int[] arr_3015) {
    for (int i_3015 : arr_3015) {
        System.out.print(i_3015 + " ");
    }
    System.out.println();
}
}

```

## B. Penjelasan

### a. Deklarasi package dan class

package pekan8\_2511533015;

public class ShellSort\_2511533015 {

# penjelasan:

- Package pekan8\_2511533015;  
➔ Menunjukkan bahwa program berada dalam package Bernama pekan8\_2511533015
- Public class shellsort\_2511533015  
➔ Mendefinisikan class utama yang berisi algoritma shell sort

### b. Method shellsort

```
public static void ShellSort_2511533015(int[] A_3015)
```

# fungsi:

Method ini digunakan untuk mengurutkan array menggunakan algoritma shell sort

^ parameter

int[] A\_3015

➔ Merupakan array yang akan diurutkan

^ menentukan Panjang array

int n\_3015 = A\_3015.length;

➔ Mengambil jumlah elemen dalam array

Misalnya:

{3,10,4,6,8,9,7,2,1,5}

Jumlah data:

^ menentukan nilai gap awal

int gap\_3015 = n\_3015 / 2;

➔ Shell sort menggunakan jarak (gap) untuk membandingkan data.

Karena:

n = 10

maka:

gap = 10 / 2

gap = 5

c. Perulangan gap

while (gap\_3015 > 0)

# fungsi:

Mengulangi proses sorting selama nilai gap masih lebih besar dari 0.

^ urutan gap yang terjadi:

5

2

1

0

Ketika gap menjadi 0, proses berhenti.

d. Perulangan data

```
for (int i_3015 = gap_3015; i_3015 < n_3015; i_3015++)
```

# fungsi:

Menelusuri data mulai dari indeks sebesar gap

Jika:

gap = 5

maka:

i = 5 sampai 9

e. Menyimpan data sementara

```
int temp_3015 = A_3015[i_3015];
```

# fungsi:

Menyimpan nilai yang akan dipindahkan

Contoh:

A[5] = 9

Maka:

temp = 9

f. Menyimpan posisi saat ini

```
int j_3015 = i_3015;
```

# fungsi:

Variable j digunakan untuk menggeser posisi data selama proses pengurutan

g. Perbandingan dan pergeseran data

```
while (j_3015 >= gap_3015 &&  
      A_3015[j_3015-gap_3015] > temp_3015)
```

# fungsi:

Membandingkan elemen yang berjarak gap.

Misalnya:

3 10 4 6 8 9 7 2 1 5

Gap = 5

Perbandingan:

3 ↔ 9

10 ↔ 7

4 ↔ 2

6 ↔ 1

8 ↔ 5

Jika data sebelumnya lebih besar:

```
A[j] = A[j-gap];
```

Data digeser ke kanan

Contoh:

$10 > 7$

Maka:

10 dipindahkan

Kemudian:

$j_{3015} = j_{3015} - \text{gap}_{3015};$

posisi mundur sejauh gap

h. Menempatkan data

$A_{3015}[j_{3015}] = \text{temp}_{3015};$

# fungsi:

Meletakkan data pada posisi yang tepat

Misalnya:

10 dipindah

7 masuk ke tempatnya

i. Mengurangi gap

$\text{gap}_{3015} = \text{gap}_{3015} / 2;$

# fungsi:

Memperkecil jarak perbandingan

Urutan:

Gap = 5

Gap = 2

Gap = 1

Gap = 0

Ketika:

gap = 1

shell sort berubah menjadi insertion sort biasa

j. Method main

```
public static void main(String[] args_3015)
```

# fungsi:

Method utama yang menjalankan program

^ membuat array

```
int[] data_3015 =  
{3,10,4,6,8,9,7,2,1,5};
```

Data awal:

3 10 4 6 8 9 7 2 1 5

^ menampilkan data sebelum sorting

```
System.out.print("Sebelum: ");  
printArray_3015(data_3015);
```

output:

Sebelum:

3 10 4 6 8 9 7 2 1 5

^ menjalankan shell sort

```
ShellSort_2511533015(data_3015);
```

Array diurutkan secara ascending (kecil ke besar)

^ menampilkan hasil sorting

```
System.out.print("Sesudah (Shell Sort): ");
```

```
printArray_3015(data_3015);
```

output:

Sesudah (Shell Sort):

1 2 3 4 5 6 7 8 9 10

k. Method print array

```
public static void printArray_3015(int[] arr_3015)
```

# fungsi:

Menampilkan seluruh isi array

^ perulangan for-each

```
for (int i_3015 : arr_3015)
```

penjelasan:

mengambil setiap elemn array satu per satu.

Misalnya:

1

2

3

4

...

^ menampilkan data

```
System.out.print(i_3015 + " ");
```

Output:

1 2 3 4 5 6 7 8 9 10

^ ilustrasi cara kerja shell sort

Data awal:

3 10 4 6 8 9 7 2 1 5

Gap=5

Bandingkan:

3 ↔ 9

10 ↔ 7

4 ↔ 2

6 ↔ 1

8 ↔ 5

Hasil:

3 7 2 1 5 9 10 4 6 8

Gap=2

Bandingkan elemen dengan jarak 2

Hasil mendekati urut:

2 1 3 4 5 6 7 8 9 10

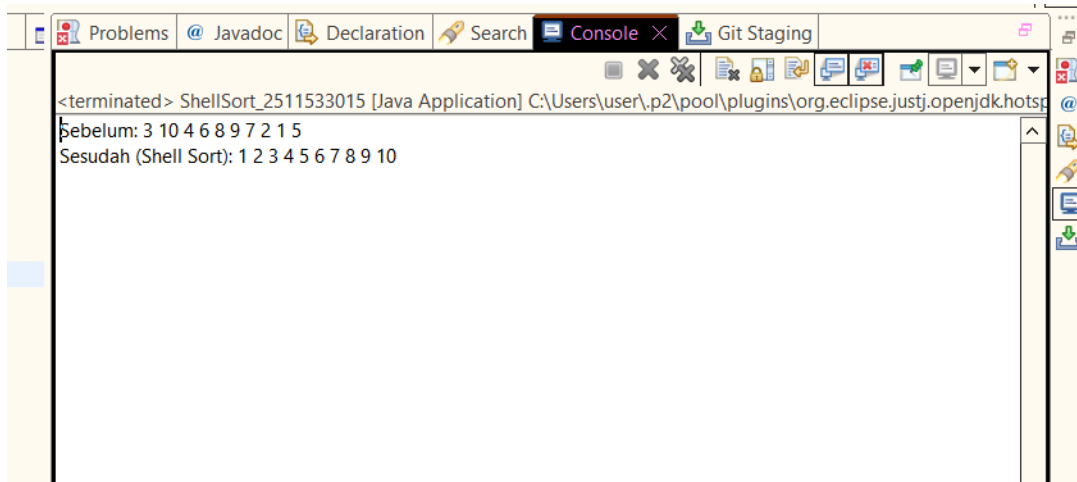
Gap=1

Menjadi insertion sort

Hasil akhir:

1 2 3 4 5 6 7 8 9 10

### C. Output program



#### 1. Kelas Quick sort

⇒ Program ini digunakan untuk mengurutkan data menggunakan algoritma quick sort. Pada program ini digunakan Teknik median of three untuk memilih pivot agar proses pengurutan lebih efisien dan mengurangi kemungkinan terjadinya kasus terburuk.

### A. Kode program

```
package pekan8_2511533015;

public class QuickSort_2511533015 {

    static void swap_3015(int[] arr_3015, int i_3015, int j_3015) {
        int temp_3015 = arr_3015[i_3015];
        arr_3015[i_3015] = arr_3015[j_3015];
        arr_3015[j_3015] = temp_3015;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_3015(int[] arr_3015, int low_3015, int high_3015) {

        int mid_3015 = low_3015 + (high_3015 - low_3015) / 2;
```

```

        // Urutkan elemen low, mid, dan high
        if (arr_3015[low_3015] > arr_3015[mid_3015]) {
            swap_3015(arr_3015, low_3015, mid_3015);
        }

        if (arr_3015[low_3015] > arr_3015[high_3015]) {
            swap_3015(arr_3015, low_3015, high_3015);
        }

        if (arr_3015[mid_3015] > arr_3015[high_3015]) {
            swap_3015(arr_3015, mid_3015, high_3015);
        }

        // Pindahkan median ke posisi high sebagai pivot
        swap_3015(arr_3015, mid_3015, high_3015);
    }

    static int partition_3015(int[] arr_3015, int low_3015, int high_3015) {

        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_3015(arr_3015, low_3015, high_3015);

        int pivot_3015 = arr_3015[high_3015];
        int i_3015 = low_3015 - 1;

        for (int j_3015 = low_3015; j_3015 <= high_3015 - 1; j_3015++) {

            // Jika elemen saat ini lebih kecil dari pivot
            if (arr_3015[j_3015] < pivot_3015) {
                i_3015++;
                swap_3015(arr_3015, i_3015, j_3015);
            }
        }

        swap_3015(arr_3015, i_3015 + 1, high_3015);
        return i_3015 + 1;
    }

    static void quickSort_3015(int[] arr_3015, int low_3015, int high_3015) {

        if (low_3015 < high_3015) {

            int pi_3015 = partition_3015(arr_3015, low_3015, high_3015);

            quickSort_3015(arr_3015, low_3015, pi_3015 - 1);
            quickSort_3015(arr_3015, pi_3015 + 1, high_3015);
        }
    }

    public static void printArr_3015(int[] arr_3015) {

        for (int i_3015 = 0; i_3015 < arr_3015.length; i_3015++) {
            System.out.print(arr_3015[i_3015] + " ");
        }

        System.out.println();
    }

    public static void main(String[] args_3015) {

        int[] arr_3015 = { 10, 7, 8, 9, 1, 5 };
        int N_3015 = arr_3015.length;
    }

```

```

        System.out.print("Data sebelum diurutkan: ");
        printArr_3015(arr_3015);

        quickSort_3015(arr_3015, 0, N_3015 - 1);

        System.out.print("Data Terurut quicksort: ");
        printArr_3015(arr_3015);
    }
}

```

## B. Penjelasan

### a. deklarasi package dan class

```

package pekan8_2511533015;

public class QuickSort_2511533015 {

```

# Penjelasan:

- Package pekan8\_2511533015  
➔ menunjukkan bahwa class berada dalam package tersebut
- Quicksort\_2511533015  
➔ Merupakan class utama yang berisi seluruh algoritma quick sort

### b. method swap\_3015()

```

static void swap_3015(int[] arr_3015, int i_3015, int j_3015)

```

# fungsi:

Menukar (swap) posisi dua elemen dalam array.

^ kode

```

int temp_3015 = arr_3015[i_3015];
arr_3015[i_3015] = arr_3015[j_3015];
arr_3015[j_3015] = temp_3015;

```

cara kerja

misal:

10 7 8 9 1 5

Tukar indeks 0 dan 4:

10  $\leftrightarrow$  1

Hasil:

1 7 8 9 10 5

Variable temp\_3015 digunakan untuk tempat penyimpanan sementara agar data tidak hilang saat proses pertukaran

c. method medianOfThree\_3015

static void medianOfThree\_3015

(int[] arr\_3015, int low\_3015, int high\_3015)

# fungsi:

Menentukan pivot menggunakan metode median of three

^ menghitung indeks Tengah

int mid\_3015 =

low\_3015 + (high\_3015 - low\_3015) / 2;

misalnya:

low = 0

high = 5

maka:

mid = 2

^ mengurutkan tiga nilai

Program membandingkan:

arr[low]  
arr[mid]  
arr[high]

^ perbandingan pertama:

if (arr\_3015[low\_3015] > arr\_3015[mid\_3015])

➔ Jika nilai awal lebih besar dari nilai Tengah maka ditukar

^ Perbandingan kedua

if (arr\_3015[low\_3015] > arr\_3015[high\_3015])

➔ Jika nilai awal lebih besar dari nilai akhir maka ditukar

^ Perbandingan ketiga

if (arr\_3015[mid\_3015] > arr\_3015[high\_3015])

➔ Jika nilai Tengah lebih besar dari akhir maka ditukar

^ Menentukan pivot

swap\_3015(arr\_3015, mid\_3015, high\_3015);

➔ Median dipindahkan ke posisi paling kanan (high) untuk dijadikan pivot

^ contoh

Data:

10 7 8 9 1 5

Yang dibandingkan:

low = 10

mid = 8

high = 5

setelah diurutkan:

5 7 8 9 1 10

Median:

8

Pivot menjadi:

8

d. method partition\_3015

static int partition\_3015

(int[] arr\_3015, int low\_3015, int high\_3015)

# fungsi:

Membagi array menjadi 2 bagian:

- Kiri -> lebih kecil dari pivot
- Kanan -> lebih besar dari pivot

^ Menentukan pivot

medianOfThree\_3015(arr\_3015, low\_3015, high\_3015);

int pivot\_3015 = arr\_3015[high\_3015];

➔ Pivot diambil dari indeks terakhir

^ menentukan posisi awal

int i\_3015 = low\_3015 - 1;

➔ Digunakan untuk melacak posisi elemen yang lebih kecil dari pivot

^ perulangan

for (int j\_3015 = low\_3015;

j\_3015 <= high\_3015 - 1;

j\_3015++)

➔ Menelusuri seluruh elemen selain pivot

^ jika lebih kecil dari pivot

```
if (arr_3015[j_3015] < pivot_3015)
```

^ geser indeks

```
i_3015++;
```

^ tukar data

```
swap_3015(arr_3015, i_3015, j_3015);
```

^ menempatkan pivot

```
swap_3015(arr_3015,
```

```
    i_3015 + 1,
```

```
    high_3015);
```

➔ Pivot ditempatkan pada posisi yang benar

^ return

```
return i_3015 + 1;
```

➔ Mengembalikan posisi pivot

e. method quicksort\_3015

```
static void quickSort_3015
```

```
(int[] arr_3015,
```

```
    int low_3015,
```

```
    int high_3015)
```

# fungsi:

Melakukan proses pengurutan menggunakan Teknik rekursif

^ kondisi rekursi

```
if (low_3015 < high_3015)
```

➔ Artinya:

Masih ada data yang perlu diurutkan

^ memanggil partition

```
int pi_3015 =  
partition_3015(arr_3015,  
                low_3015,  
                high_3015);
```

➔ Mendapatkan posisi pivot

^ rekursi kiri

```
quickSort_3015(  
arr_3015,  
low_3015,  
pi_3015 - 1);
```

➔ Mengurutkan bagian kiri pivot

^rekursi kanan

```
quickSort_3015(  
arr_3015,  
pi_3015 + 1,  
high_3015);
```

➔ Mengurutkan bagian kanan pivot

f. method printArr\_3015

```
public static void printArr_3015(int[] arr_3015)
```

# fungsi:

Menampilkan isi array

^ perulangan

```
public static void printArr_3015(int[] arr_3015)
```

➔ Menampilkan seluruh elemen

^ output

```
System.out.print(arr_3015[i_3015] + " ");
```

g. method main

```
public static void main(String[] args_3015)
```

➔ Merupakan program utama

^ membuat array

```
int[] arr_3015 =
```

```
{10, 7, 8, 9, 1, 5};
```

Data awal:

10 7 8 9 1 5

^ menghitung Panjang array

```
int N_3015 = arr_3015.length;
```

nilainya:

6

^ menampilkan data awal

```
System.out.print(
```

```
"Data sebelum diurutkan: ");
```

Output:

Data sebelum diurutkan:

10 7 8 9 1 5

^ menjalankan quick sort

```
quickSort_3015(
```

```
arr_3015,
```

```
0,
```

```
N_3015 - 1);
```

Parameter:

low = 0

high = 5

^ menampilkan hasil

low = 0

high = 5

output:

1 5 7 8 9 10

^ ilustrasi proses sorting

Data awal:

10 7 8 9 1 5

^ pivot pertama (median of three)

data yang dibandingkan:

10 , 8 , 5

Median:

8

Pivot = 8

^ setelah partition

7 1 5 8 10 9

➔ Pivot 8 sudah berada pada posisi yang benar

^ rekursi kiri

7 1 5

Menjadi:

1 5 7

^ rekursi kanan

10 9

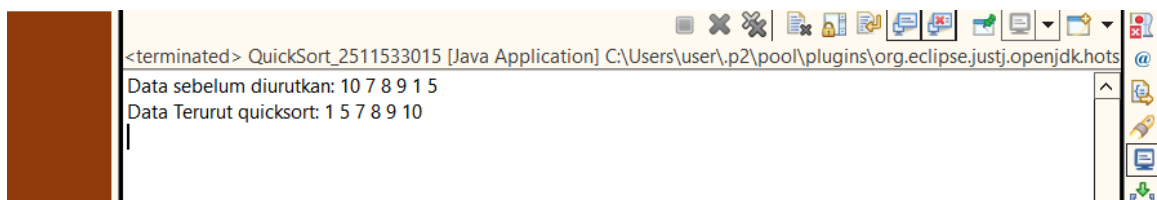
Menjadi:

9 10

^ hasil akhir

1 5 7 8 9 10

### C. Output program



```
<terminated> QuickSort_2511533015 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full\jre\bin\java.exe -Djava.library.path=C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full\jre\bin\java.exe -Xmx1024m -Xms1024m -Djava.awt.headless=true -Djava.util.logging.config.file=C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full\jre\bin\java.exe -Djava.util.logging.manager=org.eclipse.jdt.internal.ui.logging.JavaLogManager -jar C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full\jre\bin\java.exe
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

## 2. Kelas mergesortGUI\_2511533015

⇒ Program ini merupakan implementasi algoritma merge sort berbasis GUI menggunakan java swing. Berbeda dengan program merge sort biasa yang langsung menampilkan hasil akhir, program ini memperlihatkan prose pengurutan Langkah demi

Langkah (step by step) sehingga pengguna dapat memahami bagaimana algoritma merge sort bekerja.

A. Kode program

```
package pekan8_2511533015;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

import java.util.LinkedList;
import java.util.Queue;

public class MergeSortGUI_2511533015 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_3015;
```

```

private JLabel[] labelArray_3015;
private JButton stepButton_3015, resetButton_3015,
setButton_3015;

private JTextField inputField_3015;
private JPanel panelArray_3015;
private JTextArea stepArea_3015;

private Queue<int[]> mergeQueue_3015 = new
LinkedList<>();

private int stepCount_3015 = 1;
private boolean isMerging_3015 = false;
private boolean copying_3015 = false;

private int[] temp_3015;
private int left_3015, mid_3015, right_3015;
private int i_3015, j_3015, k_3015;

public MergeSortGUI_2511533015() {
    setTitle("Merge Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // panel input
    JPanel inputPanel_3015 = new JPanel(new FlowLayout());
    inputField_3015 = new JTextField(30);
    setButton_3015 = new JButton("Set Array");
    inputPanel_3015.add(new JLabel("Masukan angka
(pisahkan dengan koma)"));
    inputPanel_3015.add(inputField_3015);

```

```

inputPanel_3015.add(setButton_3015);

// panel array visual
panelArray_3015 = new JPanel();
panelArray_3015.setLayout(new FlowLayout());

// panel kontrol
JPanel controlPanel_3015 = new JPanel();
stepButton_3015 = new JButton("Langkah selanjutnya");
resetButton_3015 = new JButton("Reset");
controlPanel_3015.add(stepButton_3015);
controlPanel_3015.add(resetButton_3015);

// area text untuk log langkah langkah
stepArea_3015 = new JTextArea(8, 60);
stepArea_3015.setEditable(false);
stepArea_3015.setFont(new Font("Monospaced",
Font.PLAIN, 14));

JScrollPane scrollPane_3015 = new
JScrollPane(stepArea_3015);

// tambahkan panel ke frame
add(inputPanel_3015, BorderLayout.NORTH);
add(panelArray_3015, BorderLayout.CENTER);
add(controlPanel_3015, BorderLayout.SOUTH);
add(scrollPane_3015, BorderLayout.EAST);

// event set array
setButton_3015.addActionListener(e ->
setArrayFromInput_3015());

```

```

// event selanjutnya
stepButton_3015.addActionListener(e ->
performStep_3015());

// event reset
resetButton_3015.addActionListener(e -> reset_3015());
}

private void setArrayFromInput_3015() {
String text_3015 = inputField_3015.getText().trim();
if (text_3015.isEmpty()) return;
String[] parts_3015 = text_3015.split(",");
array_3015 = new int[parts_3015.length];
try {
    for (int k = 0; k < parts_3015.length; k++) {
        array_3015[k] =
Integer.parseInt(parts_3015[k].trim());
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Masukkan
hanya angka dengan koma!",
                                "Error",
JOptionPane.ERROR_MESSAGE);
    return;
}

labelArray_3015 = new JLabel[array_3015.length];
panelArray_3015.removeAll();
for (int k = 0; k < array_3015.length; k++) {
    labelArray_3015[k] = new
JLabel(String.valueOf(array_3015[k]));
}

```

```

        labelArray_3015[k].setFont(new Font("Arial",
Font.BOLD, 24));

        labelArray_3015[k].setOpaque(true);
        labelArray_3015[k].setBackground(Color.WHITE);

        labelArray_3015[k].setBorder(BorderFactory.createLineBor
der(Color.BLACK));

        labelArray_3015[k].setPreferredSize(new
Dimension(50, 50));

        labelArray_3015[k].setHorizontalAlignment(SwingConstant
s.CENTER);

        panelArray_3015.add(labelArray_3015[k]);
    }

    mergeQueue_3015.clear();
    generateMergeSteps_3015(0, array_3015.length - 1);
    stepButton_3015.setEnabled(true);
    stepArea_3015.setText("");
    stepCount_3015 = 1;
    isMerging_3015 = false;

    panelArray_3015.revalidate();
    panelArray_3015.repaint();
}

private void generateMergeSteps_3015(int left, int right) {
    if (left >= right) return;
    int mid = (left + right) / 2;
    generateMergeSteps_3015(left, mid);
    generateMergeSteps_3015(mid + 1, right);
}

```

```

mergeQueue_3015.add(new int[]{left, mid, right});
}

private void performStep_3015() {
    resetHighlights_3015();

    if (!isMerging_3015 && !mergeQueue_3015.isEmpty()) {
        int[] range_3015 = mergeQueue_3015.poll();
        left_3015 = range_3015[0];
        mid_3015 = range_3015[1];
        right_3015 = range_3015[2];
        temp_3015 = new int[right_3015 - left_3015 + 1];
        i_3015 = left_3015;
        j_3015 = mid_3015 + 1;
        k_3015 = 0;
        copying_3015 = false;
        isMerging_3015 = true;
        stepArea_3015.append("Langkah " +
stepCount_3015++ +
                                ": Mulai merge dari " + left_3015 + "
ke " + right_3015 + "\n");
        return;
    }

    if (isMerging_3015 && !copying_3015) {
        if (i_3015 <= mid_3015 && j_3015 <= right_3015)
        {

            labelArray_3015[i_3015].setBackground(Color.CYAN);

            labelArray_3015[j_3015].setBackground(Color.CYAN);

```

```

        if (array_3015[i_3015] <=
array_3015[j_3015]) {
            temp_3015[k_3015++] =
array_3015[i_3015++];
        } else {
            temp_3015[k_3015++] =
array_3015[j_3015++];
        }
        stepArea_3015.append("Langkah " +
stepCount_3015++ + ": Bandingkan dan salin elemen\n");
        return;
    } else if (i_3015 <= mid_3015) {
        temp_3015[k_3015++] =
array_3015[i_3015++];
        stepArea_3015.append("Langkah " +
stepCount_3015++ + ": Salin sisa kiri\n");
        return;
    } else if (j_3015 <= right_3015) {
        temp_3015[k_3015++] =
array_3015[j_3015++];
        stepArea_3015.append("Langkah " +
stepCount_3015++ + ": Salin sisa kanan\n");
        return;
    } else {
        copying_3015 = true;
        k_3015 = 0;
        return;
    }
}

if (copying_3015 && k_3015 < temp_3015.length) {

```

```

        array_3015[left_3015 + k_3015] =
temp_3015[k_3015];
        labelArray_3015[left_3015 +
k_3015].setText(String.valueOf(temp_3015[k_3015]));
        labelArray_3015[left_3015 +
k_3015].setBackground(Color.GREEN);
        k_3015++;
        stepArea_3015.append("Langkah " +
stepCount_3015++ + ": Tempelkan ke array utama\n");
        return;
    }

    if (copying_3015 && k_3015 == temp_3015.length) {
        isMerging_3015 = false;
        copying_3015 = false;
    }

    if (mergeQueue_3015.isEmpty() && !isMerging_3015) {
        stepArea_3015.append("Selesai.\n");
        stepButton_3015.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort
selesai!");
    }
}

private void resetHighlights_3015() {
    if (labelArray_3015 == null) return;
    for (JLabel label : labelArray_3015) {
        label.setBackground(Color.WHITE);
    }
}

```

```

private void reset_3015() {
    inputField_3015.setText("");
    panelArray_3015.removeAll();
    panelArray_3015.revalidate();
    panelArray_3015.repaint();
    stepArea_3015.setText("");
    stepButton_3015.setEnabled(false);
    mergeQueue_3015.clear();
    isMerging_3015 = false;
    stepCount_3015 = 1;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511533015 gui = new
MergeSortGUI_2511533015();
        gui.setVisible(true);
    });
}
}

```

## B. Penjelasan

### a. Deklarasi package dan import

```
package pekan8_2511533015;
```

➔ Menunjukkan bahwa program berada dalam package pekan8\_2511533015

```
^ library swing
```

```
import javax.swing.*;
```

➔ Digunakan untuk membuat tampilan GUI seperti:

```
➤ JFrame
```

- JButton
- JLabel
- JTextField
- JTextArea
- JOptionPane

^ library AWT

```
import javax.swing.*;
```

➔ Digunakan untuk:

- Warna (color)
- Font
- Layout
- Ukuran komponen

^ queue

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

➔ Digunakan untuk menyimpan urutan proses merge yang akan dilakukan.

b. Deklarasi class

```
public class MergeSortGUI_2511533015 extends JFrame
```

artinnya:

class ini mewarisi (extend) JFrame sehingga dapat menjadi sebuah jendela GUI

c. Deklarasi variable

^ array utama

```
private int[] array_3015;
```

➔ Digunakan untuk menyimpan data yang akan diurutkan

Contoh:

10 5 8 2 7

^ label visual array

```
private JLabel[] labelArray_3015;
```

➔ Digunakan untuk menampilkan setiap elemen array dalam bentuk kotak kotak di GUI

Contoh:

[10] [5] [8] [2] [7]

^ tombol

```
private JButton stepButton_3015;
```

```
private JButton resetButton_3015;
```

```
private JButton setButton_3015;
```

➔ Fungsinya:

Set array = memasukkan data

Langkah selanjutnya = menjalankan proses merge satu Langkah

Reset = mengulang program

^ input

```
private JTextField inputField_3015;
```

➔ Tempat pengguna memasukkan angka

Contoh:

10,5,8,2,7

^ area log

```
private JTextArea stepArea_3015;
```

➔ Menampilkan catatan proses merge sort

Contoh:

Langkah 1 : Mulai merge

Langkah 2 : Bandingkan elemen

Langkah 3 : Salin elemen

- d. Queue untuk menyimpan tahapan merge

```
private Queue<int[]> mergeQueue_3015 = new LinkedList<>();
```

Queue digunakan untuk menyimpan:

[left, mid, right]

Yang akan diproses: 0 1 3

0 1 3

Artinya:

left = 0

mid = 1

right = 3

- e. Constructor

```
public MergeSortGUI_2511533015()
```

➔ Constructor dipanggil saat objek dibuat

^ mengatur judul

```
setTitle("Merge Sort Langkah per Langkah");
```

➔ Menampilkan judul jendela

^ ukuran jendela

```
setSize(750,400);
```

ukuran:

Lebar = 750 px

Tinggi = 400 px

^ menutup program

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

jika tombol X ditekan maka program berhenti .

^ menempatkan frame di Tengah  
`setLocationRelativeTo(null);`

f. Membuat panel input

`JPanel inputPanel_3015 = new JPanel(new FlowLayout());`

Panel ini berisi:

Masukkan angka:

[

[Set Array]

g. Membuat panel array

`panelArray_3015 = new JPanel();`

➔ Digunakan untuk menampilkan kotak kotak angka

Contoh:

[10] [5] [8] [2] [7]

h. Membuat panel control

`JPanel controlPanel_3015`

Berisi tombol:

[Langkah Selanjutnya]

[Reset]

i. Membuat area log

`stepArea_3015 = new JTextArea(8,60);`

➔ Menampilkan seluruh Langkah merge sort

j. Event set array

```
setButton_3015.addActionListener(  
    e -> setArrayFromInput_3015()  
);
```

Saat tombol set array ditekan:

Input dibaca

Array dibuat

Visualisasi ditampilkan

k. Method `setArrayFromInput_3015`

➔ Method ini digunakan untuk membaca input dari pengguna

^ membaca input

```
String text_3015 =  
    inputField_3015.getText().trim();
```

misalnya:

10,5,8,2,7

^ memecah berdasarkan koma

```
String[] parts_3015 =  
    text_3015.split(",");
```

hasil:

10

5

8

2

7

^ mengubah menjadi integer

`Integer.parseInt(...)`

Sehingga:

`Integer.parseInt(...)`

^ jika input salah

`catch(NumberFormatException)`

muncul pesan:

Masukkan hanya angka dengan koma!

1. Membuat visualisasi array

`labelArray_3015[k] =  
new JLabel(...)`

setiap angka dibuat menjadi kotak

[10]

[5]

[8]

[2]

[7]

- m. Method `generateMergeSteps_3015`

`private void generateMergeSteps_3015  
(int left,int right)`

➔ Membuat seluruh tahapan merge menggunakan rekursi

Contoh:

Array:

10 5 8 2

Tahapan yang masuk queue:

[0,0,1]

[2,2,3]

[0,1,3]

n. Method performStep\_3015

➔ Ini Adalah inti program

Method ini dijalankan saat tombol

Langkah Selanjutnya

Ditekan.

^ menghapus warna sebelumnya

resetHighlights\_3015();

➔ Mengembalikan proses merge berikutnya

mergeQueue\_3015.poll();

mengambil data:

left

mid

right

contoh:

left

mid

right

^ mengambil proses merge berikutnya

mergeQueue\_3015.poll();

mengambil data:

left

mid

right

contoh:

left

mid

right

o. Membandingkan elemen

if(array\_3015[i\_3015]

<=

array\_3015[j\_3015])

proses merge dilakukan

contoh:

if(array\_3015[i\_3015]

<=

array\_3015[j\_3015])

karena:

5 < 8

Maka:

5 masuk temp

p. Highlight warna

Saat dibandingkan:

Color.CYAN

Warna biru muda menunjukkan:  
Sedang dibandingkan

Saat ditempel ke array utama:  
Color.GREEN

Artinya:  
Data sudah terurut

q. Menyalin ke array utama  
array\_3015[left\_3015+k\_3015]  
=  
temp\_3015[k\_3015];

➔ Data hasil merge dimasukkan Kembali ke array utama

r. Jika selesai  
JOptionPane.showMessageDialog

Menampilkan:  
Merge Sort selesai!

s. Method resetHighlights\_3015  
private void resetHighlights\_3015()

➔ Mengembalikan semua warna menjadi putih

Sebelum:  
[10] hijau  
[5] biru

Sesudah:

[10] putih

[5] putih

t. Method reset\_3015

➔ Digunakan saat tombol reset ditekan

Yang dilakukan:

```
inputField_3015.setText("");
```

➔ Menghapus input

```
panelArray_3015.removeAll();
```

➔ Menghapus visualisasi

```
stepArea_3015.setText("");
```

➔ Menghapus visualisasi

```
stepArea_3015.setText("");
```

➔ Menghapus log

```
mergeQueue_3015.clear();
```

➔ Mengosongkan queue

u. Method main

```
public static void main(String[] args)
```

➔ Titik awal program

^ Menjalankan GUI

```
SwingUtilities.invokeLater(...)
```

➔ Membuat gui berjalan pada event dispatch thread swing

^ Membuat objek gui

```
MergeSortGUI_2511533015 gui  
=  
new MergeSortGUI_2511533015();
```

^ menampilkan gui

```
gui.setVisible(true);
```

➔ Menampilkan jendela ke layer

^ alur kerja program

Misalnya pengguna memasukkan:

10,5,8,2

➤ Langkah 1

Array tampil:

[10] [5] [8] [2]

➤ Langkah 2

Queue merge dibuat:

[0,0,1]

[2,2,3]

[0,1,3]

➤ Langkah 3

Tekan:

Langkah Selanjutnya

Program membandingkan:

10 dan 5

➤ Langkah 4

Warna biru:

[10] [5]

➤ Langkah 5

Data hasil merge:

5 10

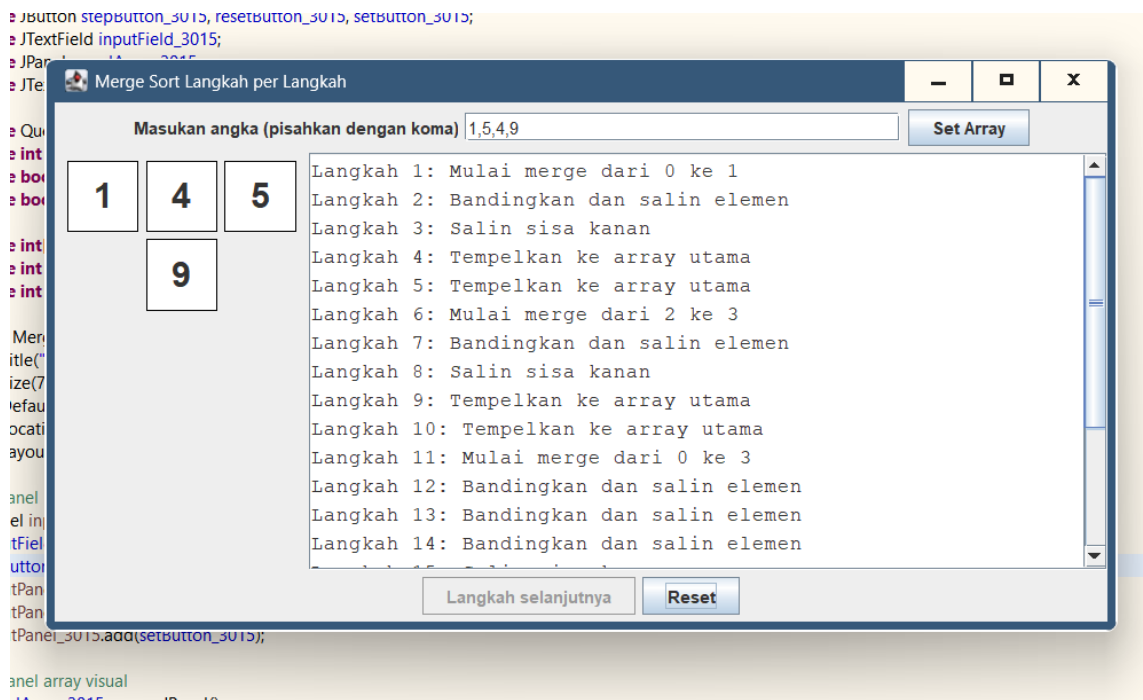
Ditandai warna hijau

➤ Langkah 6

Proses berlanjut hingga:

2 5 8 10

### C. Output program



### 3. Kelas mergesort\_2511533015

#### A. Kode program

```
package pekan8_2511533015;

public class MergeSort_2511533015 {

    void merge_3015(int arr_3015[], int l_3015, int m_3015, int r_3015) {
```

```

// Find sizes of two subarrays to be merged
int n1_3015 = m_3015 - l_3015 + 1;
int n2_3015 = r_3015 - m_3015;

/* Create temp arrays */
int L_3015[] = new int[n1_3015];
int R_3015[] = new int[n2_3015];

/* Copy data to temp arrays */
for (int i_3015 = 0; i_3015 < n1_3015; ++i_3015)
    L_3015[i_3015] = arr_3015[l_3015 + i_3015];

for (int j_3015 = 0; j_3015 < n2_3015; ++j_3015)
    R_3015[j_3015] = arr_3015[m_3015 + 1 + j_3015];

int i_3015 = 0, j_3015 = 0;

// Initial index of merged subarray array
int k_3015 = l_3015;

while (i_3015 < n1_3015 && j_3015 < n2_3015) {

    if (L_3015[i_3015] <= R_3015[j_3015]) {
        arr_3015[k_3015] = L_3015[i_3015];
        i_3015++;
    } else {
        arr_3015[k_3015] = R_3015[j_3015];
        j_3015++;
    }

    k_3015++;
}

/* Copy remaining elements of L[] if any */
while (i_3015 < n1_3015) {
    arr_3015[k_3015] = L_3015[i_3015];
    i_3015++;
    k_3015++;
}

/* Copy remaining elements of R[] if any */
while (j_3015 < n2_3015) {
    arr_3015[k_3015] = R_3015[j_3015];
    j_3015++;
    k_3015++;
}
}

void sort_3015(int arr_3015[], int l_3015, int r_3015) {

    if (l_3015 < r_3015) {

        // Find the middle point
        int m_3015 = (l_3015 + r_3015) / 2;

        // Sort first and second halves
        sort_3015(arr_3015, l_3015, m_3015);
        sort_3015(arr_3015, m_3015 + 1, r_3015);

        // Merge the sorted halves
        merge_3015(arr_3015, l_3015, m_3015, r_3015);
    }
}

```

```

}

/* A utility function to print array */
static void printArray_3015(int arr_3015[]) {

    int n_3015 = arr_3015.length;

    for (int i_3015 = 0; i_3015 < n_3015; ++i_3015)
        System.out.print(arr_3015[i_3015] + " ");

    System.out.println();
}

public static void main(String args_3015[]) {

    int arr_3015[] = { 12, 11, 13, 5, 6, 7 };

    System.out.println("Sebelum Terurut");
    printArray_3015(arr_3015);

    MergeSort_2511533015 ob_3015 = new MergeSort_2511533015();
    ob_3015.sort_3015(arr_3015, 0, arr_3015.length - 1);

    System.out.println("\nSesudah Terurut menggunakan Merge Sort");
    printArray_3015(arr_3015);
}
}

```

## B. Penjelasan

### a. Deklarasi class

```
public class MergeSort_2511533015 {
```

➔ Class utama yang berisi seluruh algoritma merge sort

### b. Method merge\_3015

```
void merge_3015(int arr_3015[], int l_3015,
                int m_3015, int r_3015)
```

➔ Menggabungkan dua bagian array yang sudah terurut menjadi satu bagian yang terurut.

Parameter:

# arr\_3015 = array utama

# l\_3015 = indeks kiri

# m\_3015 = indeks Tengah

# r\_3015 = indeks kanan

^ menghitung ukuran subarray

```
int n1_3015 = m_3015 - l_3015 + 1;
```

```
int n2_3015 = r_3015 - m_3015;
```

contoh:

```
Array = [12 11 13 5 6 7]
```

```
l = 0
```

```
m = 2
```

```
r = 5
```

maka:

```
n1 = 3
```

```
n2 = 3
```

^ membuat array sementara

```
int L_3015[] = new int[n1_3015];
```

```
int R_3015[] = new int[n2_3015];
```

➔ Digunakan untuk menyimpan data sementara

Contoh:

```
L = [12 11 13]
```

```
R = [5 6 7]
```

^ menyalin data ke array kiri

```
for (int i_3015 = 0; i_3015 < n1_3015; ++i_3015)
```

```
    L_3015[i_3015] = arr_3015[l_3015 + i_3015];
```

Hasil:

```
L = [12 11 13]
```

Menyalin data ke array kanan

```
for (int j_3015 = 0; j_3015 < n2_3015; ++j_3015)
```

```
    R_3015[j_3015] =
```

```
        arr_3015[m_3015 + 1 + j_3015];
```

hasil:

R = [5 6 7]

^ inisialisasi variable

```
int i_3015 = 0;
```

```
int j_3015 = 0;
```

```
int k_3015 = l_3015;
```

^ membandingkan dua varibel

```
while (i_3015 < n1_3015 &&
```

```
    j_3015 < n2_3015)
```

➔ Loop berjalan selama kedua array masih memiliki data

^ jika data kiri lebih kecil

```
if (L_3015[i_3015] <= R_3015[j_3015])
```

contoh:

12 dibandingkan 5

Karena:

5 lebih kecil

Maka:

```
arr[k] = R[j];
```

^ menambah indeks

k\_3015++;

➔ Berpindah ke posisi berikutnya

^ menyalin sisa data kiri

while (i\_3015 < n1\_3015)

➔ Jika array kanan habis terlebih dahulu

Contoh:

L = [11 12]

R = []

Maka:

11 dan 12 langsung disalin

^ menyalin sisa data kanan

while (j\_3015 < n2\_3015)

➔ Jika array kiri habis terlebih dahulu

Contoh:

L = []

R = [13]

Maka:

13 langsung disalin

c. Method sort\_3015

```
void sort_3015(int arr_3015[],  
               int l_3015,  
               int r_3015)
```

➔ Membagi array menjadi bagian yang lebih kecil secara rekursif

^ kondisi rekursi

if (l\_3015 < r\_3015)

artinya:

masih ada lebih dari satu elemen

^ menentukan titik Tengah

int m\_3015 = (l\_3015 + r\_3015) / 2;

contoh:

l = 0

r = 5

maka:

m = 2

^ membagi bagian kiri

```
sort_3015(arr_3015,  
          l_3015,  
          m_3015);
```

contoh:

[12 11 13]

^ membagi bagian kanan

```
sort_3015(arr_3015,  
          m_3015 + 1,  
          r_3015);
```

contoh:

[5 6 7]

^ menggabungkan Kembali

```
merge_3015(arr_3015,  
            l_3015,  
            m_3015,  
            r_3015);
```

➔ Setelah kiri dan kanan terurut

^ ilustrasi rekursi

Data awal:

12 11 13 5 6 7

^ pembagian pertama

12 11 13 | 5 6 7

^ pembagian kedua

12 | 11 13

5 | 6 7

Pembagian ketiga

12

11

13

5

6

7

➔ Semua sudah menjadi satu elemen

^ proses merge

Gabung:

11 + 13

Hasil;

11 13

Gabung:

12 + (11 13)

Hasil:

11 12 13

Gabung:

6 + 7

Hasil:

6 7

Gabung:

5 + (6 7)

Hasil;

5 6 7

Merge terakhir:

11 12 13

5 6 7

Hasil:

5 6 7 11 12 13

d. Method `printArray_3015`

`static void printArray_3015(int arr_3015[])`

➔ Menampilkan seluruh isi array

^ menghitung Panjang array  
`int n_3015 = arr_3015.length;`

^ menampilkan isi array  
`for (int i_3015 = 0;  
    i_3015 < n_3015;  
    ++i_3015)`

Output:

12 11 13 5 6 7

Atau:

5 6 7 11 12 13

e. Method main

```
public static void main(String args_3015[])
```

➔ Titik awal program

^ membuat array  
`int arr_3015[] =  
{12,11,13,5,6,7};`

Data awal:

12 11 13 5 6 7

^ menampilkan sebelum sorting  
`System.out.println("Sebelum Terurut");`

Output:

Sebelum Terurut

12 11 13 5 6 7

^ membuat objek

```
MergeSort_2511533015 ob_3015 =  
new MergeSort_2511533015();
```

➔ Karena method sort\_3015 () bukan static, maka harus dibuat objek terlebih dahulu

^ menjalankan merge sort

```
ob_3015.sort_3015(  
arr_3015,  
0,  
arr_3015.length - 1);
```

parameter:

```
array = arr_3015
```

```
l = 0
```

```
r = 5
```

^ menampilkan hasil:

```
System.out.println(  
"Sesudah Terurut menggunakan Merge Sort");
```

Output:

Sesudah Terurut menggunakan Merge Sort

5 6 7 11 12 13

^ simulasi lengkap

Sebelum:

12 11 13 5 6 7

Split:

12 11 13 | 5 6 7

Split lagi

12 | 11 13

5 | 6 7

Menjadi:

12

11

13

5

6

7

Merge:

11 13

Merge:

11 12 13

Merge:

6 7

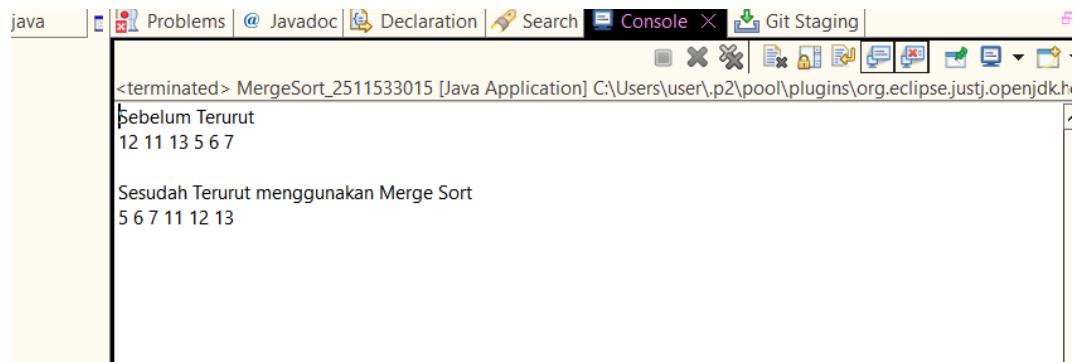
Merge:

5 6 7

Merge terakhir:

5 6 7 11 12 13

C. Output program



#### 4. Kelas bubblesortGUI\_2511533015

⇒ Merupakan implementasi algoritma bubble sort dalam bentuk GUI menggunakan java swing. Program memungkinkan pengguna memasukkan data angka, kemudian melihat proses pengurutan bubble sort Langkah demi langkah.

##### A. Kode program

```
package pekan8_2511533015;  
  
import java.awt.BorderLayout;  
import java.awt.Color;  
import java.awt.Dimension;  
import java.awt.FlowLayout;  
import java.awt.Font;  
  
import javax.swing.BorderFactory;  
import javax.swing.JButton;  
import javax.swing.JFrame;  
import javax.swing.JLabel;  
import javax.swing.JOptionPane;  
import javax.swing.JPanel;  
import javax.swing.JScrollPane;  
import javax.swing.JTextArea;
```

```

import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class BubbleSortGUI_2511533015 extends JFrame {

    private static final long serialVersionUID = 1L;

    private int[] array_3015;
    private JLabel[] labelArray_3015;

    private JTextField inputField_3015;
    private JTextArea stepArea_3015;

    private JButton setButton_3015;
    private JButton stepButton_3015;
    private JButton resetButton_3015;

    private JPanel panelArray_3015;

    private int i_3015 = 0;
    private int j_3015 = 0;
    private int stepCount_3015 = 1;
    private boolean sorting_3015 = false;

    public BubbleSortGUI_2511533015() {

        setTitle("Bubble Sort GUI 2511533015");
        setSize(900, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

```

```

setLayout(new BorderLayout());

JPanel inputPanel_3015 = new JPanel(new FlowLayout());

inputField_3015 = new JTextField(25);
setButton_3015 = new JButton("Set Array");

inputPanel_3015.add(new JLabel("Input angka (pisahkan koma:"));
inputPanel_3015.add(inputField_3015);
inputPanel_3015.add(setButton_3015);

panelArray_3015 = new JPanel(new FlowLayout());

JPanel controlPanel_3015 = new JPanel();

stepButton_3015 = new JButton("Langkah Selanjutnya");
resetButton_3015 = new JButton("Reset");

stepButton_3015.setEnabled(false);

controlPanel_3015.add(stepButton_3015);
controlPanel_3015.add(resetButton_3015);

stepArea_3015 = new JTextArea(12, 40);
stepArea_3015.setEditable(false);

JScrollPane scrollPane_3015 = new JScrollPane(stepArea_3015);

add(inputPanel_3015, BorderLayout.NORTH);
add(panelArray_3015, BorderLayout.CENTER);
add(controlPanel_3015, BorderLayout.SOUTH);

```

```

add(scrollPane_3015, BorderLayout.EAST);

setButton_3015.addActionListener(e -> setArrayFromInput_3015());
stepButton_3015.addActionListener(e -> performStep_3015());
resetButton_3015.addActionListener(e -> reset_3015());
}

private void setArrayFromInput_3015() {

    String text_3015 = inputField_3015.getText().trim();

    if (text_3015.isEmpty())
        return;

    try {

        String[] parts_3015 = text_3015.split(",");

        array_3015 = new int[parts_3015.length];

        for (int k_3015 = 0; k_3015 < parts_3015.length; k_3015++) {
            array_3015[k_3015] = Integer.parseInt(parts_3015[k_3015].trim());
        }

        panelArray_3015.removeAll();

        labelArray_3015 = new JLabel[array_3015.length];

        for (int k_3015 = 0; k_3015 < array_3015.length; k_3015++) {

```

```

        labelArray_3015[k_3015] = new
JLabel(String.valueOf(array_3015[k_3015]));

        labelArray_3015[k_3015].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_3015[k_3015].setOpaque(true);
        labelArray_3015[k_3015].setBackground(Color.WHITE);

labelArray_3015[k_3015].setBorder(BorderFactory.createLineBorder(Color.BLACK
));

        labelArray_3015[k_3015].setPreferredSize(new Dimension(60, 60));

labelArray_3015[k_3015].setHorizontalAlignment(SwingConstants.CENTER);

        panelArray_3015.add(labelArray_3015[k_3015]);
    }

    i_3015 = 0;
    j_3015 = 0;
    stepCount_3015 = 1;
    sorting_3015 = true;

    stepArea_3015.setText("");
    stepButton_3015.setEnabled(true);

    panelArray_3015.revalidate();
    panelArray_3015.repaint();

} catch (NumberFormatException ex_3015) {

JOptionPane.showMessageDialog(
    this,

```

```

        "Masukkan angka yang dipisahkan koma!",
        "Error",
        JOptionPane.ERROR_MESSAGE);
    }
}

private void performStep_3015() {

    if (!sorting_3015 || i_3015 >= array_3015.length - 1) {

        sorting_3015 = false;
        stepButton_3015.setEnabled(false);

        JOptionPane.showMessageDialog(this,
            "Bubble Sort selesai!");

        return;
    }

    resetHighlights_3015();

    labelArray_3015[j_3015].setBackground(Color.CYAN);
    labelArray_3015[j_3015 + 1].setBackground(Color.CYAN);

    StringBuilder log_3015 = new StringBuilder();

    if (array_3015[j_3015] > array_3015[j_3015 + 1]) {

        int temp_3015 = array_3015[j_3015];
        array_3015[j_3015] = array_3015[j_3015 + 1];
        array_3015[j_3015 + 1] = temp_3015;
    }
}

```

```

updateLabels_3015();

labelArray_3015[j_3015].setBackground(Color.RED);
labelArray_3015[j_3015 + 1].setBackground(Color.RED);

log_3015.append("Langkah ")
    .append(stepCount_3015)
    .append(": Tukar ")
    .append(array_3015[j_3015 + 1])
    .append(" dengan ")
    .append(array_3015[j_3015])
    .append("\n");

} else {

    log_3015.append("Langkah ")
        .append(stepCount_3015)
        .append(": Tidak ada pertukaran\n");
}

log_3015.append("Array : ")
    .append(arrayToString_3015(array_3015))
    .append("\n\n");

stepArea_3015.append(log_3015.toString());

j_3015++;

if (j_3015 >= array_3015.length - i_3015 - 1) {
    j_3015 = 0;

```

```

        i_3015++;
    }

    stepCount_3015++;
}

private void updateLabels_3015() {

    for (int k_3015 = 0; k_3015 < array_3015.length; k_3015++) {
        labelArray_3015[k_3015]
            .setText(String.valueOf(array_3015[k_3015]));
    }
}

private void resetHighlights_3015() {

    for (JLabel label_3015 : labelArray_3015) {
        label_3015.setBackground(Color.WHITE);
    }
}

private void reset_3015() {

    inputField_3015.setText("");
    panelArray_3015.removeAll();
    panelArray_3015.revalidate();
    panelArray_3015.repaint();

    stepArea_3015.setText("");

    i_3015 = 0;

```

```

j_3015 = 0;
stepCount_3015 = 1;

sorting_3015 = false;

stepButton_3015.setEnabled(false);
}

private String arrayToString_3015(int[] arr_3015) {

    StringBuilder sb_3015 = new StringBuilder();

    for (int k_3015 = 0; k_3015 < arr_3015.length; k_3015++) {

        sb_3015.append(arr_3015[k_3015]);

        if (k_3015 < arr_3015.length - 1) {
            sb_3015.append(", ");
        }
    }

    return sb_3015.toString();
}

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {

        BubbleSortGUI_2511533015 gui_3015 =
            new BubbleSortGUI_2511533015();

```

```

        gui_3015.setVisible(true);
    });
}
} package pekan8_2511533015;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class BubbleSortGUI_2511533015 extends JFrame {

    private static final long serialVersionUID = 1L;

    private int[] array_3015;
    private JLabel[] labelArray_3015;

    private JTextField inputField_3015;

```

```

private JTextArea stepArea_3015;

private JButton setButton_3015;
private JButton stepButton_3015;
private JButton resetButton_3015;

private JPanel panelArray_3015;

private int i_3015 = 0;
private int j_3015 = 0;
private int stepCount_3015 = 1;
private boolean sorting_3015 = false;

public BubbleSortGUI_2511533015() {

    setTitle("Bubble Sort GUI 2511533015");
    setSize(900, 450);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    JPanel inputPanel_3015 = new JPanel(new FlowLayout());

    inputField_3015 = new JTextField(25);
    setButton_3015 = new JButton("Set Array");

    inputPanel_3015.add(new JLabel("Input angka (pisahkan koma:"));
    inputPanel_3015.add(inputField_3015);
    inputPanel_3015.add(setButton_3015);

    panelArray_3015 = new JPanel(new FlowLayout());

```

```

JPanel controlPanel_3015 = new JPanel();

stepButton_3015 = new JButton("Langkah Selanjutnya");
resetButton_3015 = new JButton("Reset");

stepButton_3015.setEnabled(false);

controlPanel_3015.add(stepButton_3015);
controlPanel_3015.add(resetButton_3015);

stepArea_3015 = new JTextArea(12, 40);
stepArea_3015.setEditable(false);

JScrollPane scrollPane_3015 = new JScrollPane(stepArea_3015);

add(inputPanel_3015, BorderLayout.NORTH);
add(panelArray_3015, BorderLayout.CENTER);
add(controlPanel_3015, BorderLayout.SOUTH);
add(scrollPane_3015, BorderLayout.EAST);

setButton_3015.addActionListener(e -> setArrayFromInput_3015());
stepButton_3015.addActionListener(e -> performStep_3015());
resetButton_3015.addActionListener(e -> reset_3015());
}

private void setArrayFromInput_3015() {

String text_3015 = inputField_3015.getText().trim();

if (text_3015.isEmpty())

```

```

        return;

    try {

        String[] parts_3015 = text_3015.split(",");

        array_3015 = new int[parts_3015.length];

        for (int k_3015 = 0; k_3015 < parts_3015.length; k_3015++) {
            array_3015[k_3015] = Integer.parseInt(parts_3015[k_3015].trim());
        }

        panelArray_3015.removeAll();

        labelArray_3015 = new JLabel[array_3015.length];

        for (int k_3015 = 0; k_3015 < array_3015.length; k_3015++) {

            labelArray_3015[k_3015] = new
JLabel(String.valueOf(array_3015[k_3015]));

            labelArray_3015[k_3015].setFont(new Font("Arial", Font.BOLD, 24));
            labelArray_3015[k_3015].setOpaque(true);
            labelArray_3015[k_3015].setBackground(Color.WHITE);

            labelArray_3015[k_3015].setBorder(BorderFactory.createLineBorder(Color.BLACK
));

            labelArray_3015[k_3015].setPreferredSize(new Dimension(60, 60));

            labelArray_3015[k_3015].setHorizontalAlignment(SwingConstants.CENTER);

```

```

        panelArray_3015.add(labelArray_3015[k_3015]);
    }

    i_3015 = 0;
    j_3015 = 0;
    stepCount_3015 = 1;
    sorting_3015 = true;

    stepArea_3015.setText("");
    stepButton_3015.setEnabled(true);

    panelArray_3015.revalidate();
    panelArray_3015.repaint();

} catch (NumberFormatException ex_3015) {

    JOptionPane.showMessageDialog(
        this,
        "Masukkan angka yang dipisahkan koma!",
        "Error",
        JOptionPane.ERROR_MESSAGE);
}
}

private void performStep_3015() {

    if (!sorting_3015 || i_3015 >= array_3015.length - 1) {

        sorting_3015 = false;
        stepButton_3015.setEnabled(false);
    }
}

```

```

OptionPane.showMessageDialog(this,
    "Bubble Sort selesai!");

return;
}

resetHighlights_3015();

labelArray_3015[j_3015].setBackground(Color.CYAN);
labelArray_3015[j_3015 + 1].setBackground(Color.CYAN);

StringBuilder log_3015 = new StringBuilder();

if (array_3015[j_3015] > array_3015[j_3015 + 1]) {

    int temp_3015 = array_3015[j_3015];
    array_3015[j_3015] = array_3015[j_3015 + 1];
    array_3015[j_3015 + 1] = temp_3015;

    updateLabels_3015();

    labelArray_3015[j_3015].setBackground(Color.RED);
    labelArray_3015[j_3015 + 1].setBackground(Color.RED);

    log_3015.append("Langkah ")
        .append(stepCount_3015)
        .append(": Tukar ")
        .append(array_3015[j_3015 + 1])
        .append(" dengan ")
        .append(array_3015[j_3015])
        .append("\n");
}

```

```

    } else {

        log_3015.append("Langkah ")
            .append(stepCount_3015)
            .append(": Tidak ada pertukaran\n");
    }

    log_3015.append("Array : ")
        .append(arrayToString_3015(array_3015))
        .append("\n\n");

    stepArea_3015.append(log_3015.toString());

    j_3015++;

    if (j_3015 >= array_3015.length - i_3015 - 1) {
        j_3015 = 0;
        i_3015++;
    }

    stepCount_3015++;
}

private void updateLabels_3015() {

    for (int k_3015 = 0; k_3015 < array_3015.length; k_3015++) {
        labelArray_3015[k_3015]
            .setText(String.valueOf(array_3015[k_3015]));
    }
}

```

```

private void resetHighlights_3015() {

    for (JLabel label_3015 : labelArray_3015) {
        label_3015.setBackground(Color.WHITE);
    }
}

private void reset_3015() {

    inputField_3015.setText("");
    panelArray_3015.removeAll();
    panelArray_3015.revalidate();
    panelArray_3015.repaint();

    stepArea_3015.setText("");

    i_3015 = 0;
    j_3015 = 0;
    stepCount_3015 = 1;

    sorting_3015 = false;

    stepButton_3015.setEnabled(false);
}

private String arrayToString_3015(int[] arr_3015) {

    StringBuilder sb_3015 = new StringBuilder();

    for (int k_3015 = 0; k_3015 < arr_3015.length; k_3015++) {

```

```

        sb_3015.append(arr_3015[k_3015]);

        if (k_3015 < arr_3015.length - 1) {
            sb_3015.append(", ");
        }
    }

    return sb_3015.toString();
}

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {

        BubbleSortGUI_2511533015 gui_3015 =
            new BubbleSortGUI_2511533015();

        gui_3015.setVisible(true);
    });
}
}

```

## B. Penjelasan

### a. Import library

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

```

➔ Digunakan untuk mengatur tampilan gui seperti warna, ukuran, font dan layout

```
import javax.swing.*;
```

➔ Digunakan untuk membuat komponen GUI seperti:

- JFrame
- JLabel
- JButton
- JTextField
- JTextarea
- JScrollPane
- Joptionpane

b. Deklarasi kelas

```
public class BubbleSortGUI_2511533015 extends JFrame
```

kelas ini mewarisi (extends) kelas JFrame

artinya:

program ini Adalah sebuah jendela aplikasi GUI

c. Deklarasi variable

^ array data

```
private int[] array_3015;
```

➔ Menyimpan data angka yang akan diurutkan

Contoh:

5,3,8,1

Akan disimpan menjadi:

```
array_3015 = {5,3,8,1};
```

label visual array

```
private JLabel[] labelArray_3015;
```

➔ Digunakan untuk menampilkan setiap angka dalam bentuk kotak kotak

Contoh:

[5] [3] [8] [1]

Komponen input

```
private JTextField inputField_3015;
```

➔ Tempat pengguna memasukkan angka\

Contoh:

**10,5,7,2**

Area log

```
private JTextArea stepArea_3015;
```

➔ Digunakan untuk mencatat setiap langkah bubble sort

Contoh:

Langkah 1 : Tukar 10 dengan 5

Array : 5,10,7,2

Tombol:

```
private JButton setButton_3015;
```

```
private JButton stepButton_3015;
```

```
private JButton resetButton_3015;
```

set array:

menyimpan data input ke array:

Langkah selanjutnya:

Menjalankan satu Langkah bubble sort

Reset

Mengembalikan program ke kondisi awal

d. Variabel control bubble sort

```
private int i_3015 = 0;
```

```
private int j_3015 = 0;
```

➔ Digunakan sebagai indeks bubble sort

I = Menunjukkan putaran luar

Contoh:

Putaran 1

Putaran 2

Putaran 3

J = menunjukkan perbandingan elemen

Contoh:

Bandingkan data ke-0 dan ke-1

Bandingkan data ke-1 dan ke-2

Step counter:

```
private int stepCount_3015 = 1;
```

➔ Menghitung jumlah Langkah

Status sorting

```
private boolean sorting_3015 = false;
```

➔ Menandakan apakah proses sorting sedang berjalan

e. Constructor

```
public BubbleSortGUI_2511533015()
```

➔ Constructor dijalankan saat objek dibuat

Tugasnya:

```
# membuat jendela gui
```

```
# menambahkan panel
```

```
# menambahkan tombol
```

```
# menambahkan event
```

```
^ judul window
```

```
setTitle("Bubble Sort GUI 2511533015");
```

^ ukuran window

```
setSize(900, 450);
```

lebar:

900 px

Tinggi:

450 px

^ menempatkan window di Tengah

```
setLocationRelativeTo(null);
```

f. Event tombol

^ tombol set array

```
setButton_3015.addActionListener(  
    e -> setArrayFromInput_3015()  
);
```

^ Ketika tombol ditekan:

Set Array

^ maka fungsi:

```
setArrayFromInput_3015()
```

➔ Akan dijalankan

^ tombol step

```
stepButton_3015.addActionListener(  
    e -> performStep_3015()  
);
```

➔ Menjalankan satu Langkah bubble sort

^ tombol reset

```
resetButton_3015.addActionListener(  
    e -> reset_3015()  
);
```

➔ Menghapus semua data

g. Method `setarrayfrominput_3015()`

Method ini bertugas:

1. Membaca input

```
String text_3015 = inputField_3015.getText().trim();
```

Misalnya user mengetik:

7,4,2,9

2. Memecah data

```
String[] parts_3015 = text_3015.split(",");
```

Menjadi:

```
{"7","4","2","9"}
```

3. Mengubah ke integer

```
array_3015[k] =  
Integer.parseInt(parts_3015[k].trim());
```

Hasil:

```
{7,4,2,9}
```

4. Membuat label visual

```
labelArray_3015[k] = new JLabel(...)
```

menampilkan:

[7] [4] [2] [9]

5. Mengaktifkan tombol step

stepButton\_3015.setEnabled(true);

h. Method performstep\_3015

➔ Ini Adalah bagian utama bubble sort

^ mengecek apakah sorting selsai:

```
if (!sorting_3015 ||  
    i_3015 >= array_3015.length - 1)
```

^ jika selesai:

JOptionPane.showMessageDialog(...)

^ menampilkan:

Bubble Sort selesai!

i. Highlight data yang dibandingkan

```
labelArray_3015[j_3015]  
.setBackground(Color.CYAN);
```

```
labelArray_3015[j_3015 + 1]  
.setBackground(Color.CYAN);
```

^ warna biru menandakan:

Sedang dibandingkan

Contoh:

[5] [3] [8] [1]  
↑ ↑

j. Proses bubble sort

```
if (array_3015[j_3015]  
> array_3015[j_3015+1])
```

Jika kiri lebih besar daripada kanan:

$5 > 3$

Maka diperlukan pertukaran

^ pertukaran data

```
int temp_3015 =  
array_3015[j_3015];
```

```
array_3015[j_3015]  
= array_3015[j_3015+1];
```

```
array_3015[j_3015+1]  
= temp_3015;
```

Contoh:

Sebelum:

5 3

Sesudah:

3 5

k. Update tampilan

```
updateLabels_3015();
```

➔ Mengubah isi label agar sesuai dengan isi array terbaru

l. Highlight pertukaran

```
labelArray_3015[j_3015]  
    .setBackground(Color.RED);
```

```
labelArray_3015[j_3015+1]  
    .setBackground(Color.RED);
```

Merah berarti:

Data ditukar

m. Menampilkan log

Contoh:

```
stepArea_3015.append(...)
```

output:

Langkah 1 :

Tukar 5 dengan 3

Array :

3,5,8,1

n. Pindah ke perbandingan berikutnya

```
j_3015++;
```

misal:

Bandingkan 5 dan 3

Setelah selesai:

Bandingkan 3 dan 8

o. Pindah putaran

```
if (j_3015 >= array_3015.length - i_3015 - 1)
```

jika satu putaran selesai:

```
j_3015 = 0;
```

```
i_3015++;
```

contoh:

putaran 1:

5 3 8 1

Putaran 2:

3 5 1 8

p. Method `updateLabels_3015`

```
private void updateLabels_3015()
```

➔ Memperbaharui tampilan angka setelah pertukaran

Contoh:

Sebelum:

[5] [3]

Sesudah:

[3] [5]

q. Method `resetHighlights_3015`

```
private void resetHighlights_3015()
```

➔ Mengembalikan semua warna label menjadi putih

```
label.setBackground(Color.WHITE);
```

r. Method `reset_305+15`

➔ Menghapus seluruh isi program

^ mengosongkan input

```
inputField_3015.setText("");
```

^ menghapus array visual

```
panelArray_3015.removeAll();
```

^ menghapus log

```
stepArea_3015.setText("");
```

^ mengembalikan variable

```
i_3015 = 0;
```

```
j_3015 = 0;
```

```
stepCount_3015 = 1;
```

s. Method arraytoString\_3015

➔ Mengubah array menjadi string

Contoh:

```
{5,3,8,1}
```

Menjadi:

```
5, 3, 8, 1
```

Untuk ditampilkan pada log

t. Method main ()

```
public static void main(String[] args)
```

titik awal program

SwingUtilities.invokeLater(...)

➔ Digunakan agar gui dijalankan pada event dispatch thread (EDT), sesuai standar java swing

Kemudian:

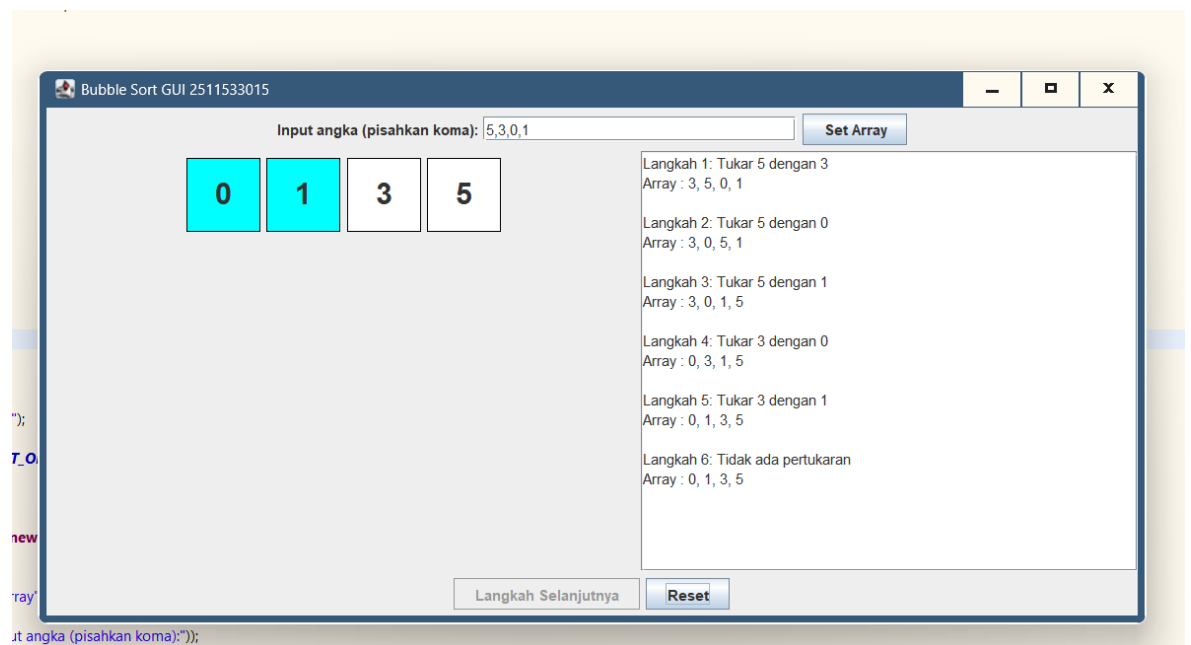
```
BubbleSortGUI_2511533015 gui =  
new BubbleSortGUI_2511533015();
```

membuat objek GUI

```
gui.setVisible(true);
```

➔ Menampilkan jendela program

### C. Output program



## BAB III

### KESIMPULAN

#### 3.1 Ringkasan hasil praktikum

Hasil praktikum menunjukkan bahwa semua program berhasil mengurutkan data dengan benar. Shell Sort menghasilkan urutan ascending dari data awal 3, 10, 4, 6, 8, 9, 7, 2, 1, 5 menjadi 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. Quick Sort juga berhasil mengurutkan data 10, 7, 8, 9, 1, 5 menjadi 1, 5, 7, 8, 9, 10. Merge Sort mengubah data 12, 11, 13, 5, 6, 7 menjadi 5, 6, 7, 11, 12, 13. Pada Bubble Sort GUI, pengguna dapat melihat perubahan warna dan langkah pertukaran hingga data tersusun rapi..

#### 3.2 Tujuan Praktikum

Program dapat dikembangkan dengan menambahkan pilihan urutan naik dan turun, membandingkan waktu eksekusi tiap algoritma, serta mempercantik tampilan GUI agar lebih interaktif. Untuk Bubble Sort GUI dan Merge Sort GUI, bisa ditambahkan animasi yang lebih halus, penanda indeks aktif, dan tombol untuk mengulang langkah tertentu. Untuk program console, dapat ditambah input dinamis dari pengguna agar lebih fleksibel.

## DAFTAR PUSTAKA

1. Oracle. *Java Platform Standard Edition Documentation*.
2. GeeksforGeeks. *Shell Sort Data Structure*.
3. GeeksforGeeks. *Quick Sort Algorithm*.
4. GeeksforGeeks. *Merge Sort Algorithm*.
5. GeeksforGeeks. *Bubble Sort Algorithm*.
6. Materi praktikum Struktur Data kelas Pekan 8, file kode Shell Sort, Quick Sort, Merge Sort, dan Bubble Sort GUI yang kamu kirim.

# TUGAS

## 1. Kelas lagu\_2511533015

### a. Kode program

```
package tugaspekan8_2511533015;

public class Lagu_2511533015 {
    private String judul_3015;
    private String penyanyi_3015;
    private int durasi_3015; // dalam detik

    public Lagu_2511533015(String judul_3015, String penyanyi_3015, int durasi_3015) {
        this.judul_3015 = judul_3015;
        this.penyanyi_3015 = penyanyi_3015;
        this.durasi_3015 = durasi_3015;
    }

    public String getJudul_3015() {
        return judul_3015;
    }

    public String getPenyanyi_3015() {
        return penyanyi_3015;
    }

    public int getDurasi_3015() {
        return durasi_3015;
    }
}
```

### b. Penjelasan

Lagu\_2511533015 adalah class untuk menyimpan data satu lagu, yaitu judul, penyanyi, dan durasi. Constructor dipakai untuk mengisi ketiga data tersebut saat objek dibuat.

Getter digunakan untuk mengambil nilai masing masing atribut

### c. Output

```

<terminated> Sorting_2511533015 [Java Application] C:\Users\user\.p2\pool\plugins\org.e
) === Sorting Playlist NIM: 2511533015 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
Pilihan: 1
Pada program ini hanya Quick Sort yang diimplementasikan.
Silakan pilih 2.

=== Sorting Playlist NIM: 2511533015 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
Pilihan: 2
Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Bella Ciao - 214 detik
5. Volare - 205 detik
6. O Sole Mio - 242 detik
7. Con Te Partiro - 240 detik

Data Setelah Quick Sort (Durasi Asc):
1. La Perla - 196 detik
2. Volare - 205 detik
3. Bella Ciao - 214 detik
4. Con Te Partiro - 240 detik
5. O Sole Mio - 242 detik
6. La Rumba Del Perdon - 252 detik
7. Mio Cristo Piange Diamanti - 270 detik

```

## 2. Classs sorting\_2511533015

### a. Kode program

```

package tugaspekan8_2511533015;

import java.util.Scanner;

public class Sorting_2511533015 {

    private static final int MAKS_DATA_3015 = 20;
    private Lagu_2511533015[] dataLagu_3015 = new Lagu_2511533015[MAKS_DATA_3015];
    private int jumlahData_3015 = 0;

    public void inputData_3015() {
        tambahLagu_3015("Mio Cristo Piange Diamanti", "Unknown", 270);
        tambahLagu_3015("La Rumba Del Perdon", "Unknown", 252);
    }
}

```

```

    tambahLagu_3015("La Perla", "Unknown", 196);
    tambahLagu_3015("Bella Ciao", "Unknown", 214);
    tambahLagu_3015("Volare", "Unknown", 205);
    tambahLagu_3015("O Sole Mio", "Unknown", 242);
    tambahLagu_3015("Con Te Partiro", "Unknown", 240);
}

private void tambahLagu_3015(String judul_3015, String penyanyi_3015, int durasi_3015) {
    if (jumlahData_3015 < MAKS_DATA_3015) {
        dataLagu_3015[jumlahData_3015] = new Lagu_2511533015(judul_3015, penyanyi_3015, durasi_3015);
        jumlahData_3015++;
    }
}

public void tampilData_3015(String judulTampilan_3015) {
    System.out.println(judulTampilan_3015);
    for (int i_3015 = 0; i_3015 < jumlahData_3015; i_3015++) {
        System.out.println((i_3015 + 1) + ". "
            + dataLagu_3015[i_3015].getJudul_3015()
            + " - "
            + dataLagu_3015[i_3015].getDurasi_3015()
            + " detik");
    }
}

public void pilihAlgoritma_3015(Scanner input_3015) {
    int pilihan_3015;

    do {
        System.out.println("=== Sorting Playlist NIM: 2511533015 ===");
        System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
        System.out.print("Pilihan: ");
        pilihan_3015 = input_3015.nextInt();

        if (pilihan_3015 != 2) {
            System.out.println("Pada program ini hanya Quick Sort yang diimplementasikan.");
            System.out.println("Silakan pilih 2.\n");
        }
    } while (pilihan_3015 != 2);

    tampilData_3015("Data Sebelum Sorting:");

    quickSort_3015(0, jumlahData_3015 - 1);

    tampilData_3015("\nData Setelah Quick Sort (Durasi Asc):");
}

private void quickSort_3015(int kiri_3015, int kanan_3015) {
    if (kiri_3015 < kanan_3015) {
        int pivot_3015 = partition_3015(kiri_3015, kanan_3015);
        quickSort_3015(kiri_3015, pivot_3015 - 1);
        quickSort_3015(pivot_3015 + 1, kanan_3015);
    }
}

```

```

}

private int partition_3015(int kiri_3015, int kanan_3015) {
    Lagu_2511533015 pivot_3015 = dataLagu_3015[kanan_3015];
    int i_3015 = kiri_3015 - 1;

    for (int j_3015 = kiri_3015; j_3015 < kanan_3015; j_3015++) {
        if (bandingkan_3015(dataLagu_3015[j_3015], pivot_3015) <= 0) {
            i_3015++;
            swap_3015(i_3015, j_3015);
        }
    }

    swap_3015(i_3015 + 1, kanan_3015);
    return i_3015 + 1;
}

private void swap_3015(int i_3015, int j_3015) {
    Lagu_2511533015 temp_3015 = dataLagu_3015[i_3015];
    dataLagu_3015[i_3015] = dataLagu_3015[j_3015];
    dataLagu_3015[j_3015] = temp_3015;
}

private int bandingkan_3015(Lagu_2511533015 a_3015, Lagu_2511533015 b_3015) {
    int hasilDurasi_3015 = Integer.compare(a_3015.getDurasi_3015(), b_3015.getDurasi_3015());
    if (hasilDurasi_3015 != 0) {
        return hasilDurasi_3015;
    }
    return a_3015.getJudul_3015().compareToIgnoreCase(b_3015.getJudul_3015());
}

public static void main(String[] args_3015) {
    Scanner input_3015 = new Scanner(System.in);

    Sorting_2511533015 program_3015 = new Sorting_2511533015();
    program_3015.inputData_3015();
    program_3015.pilihAlgoritma_3015(input_3015);

    input_3015.close();
}
}

```

## b. Penjelasan

program ini menyimpan data lagu dalam array, lalu mengurutkannya memakai Quick Sort berdasarkan durasi dari yang paling pendek ke paling panjang. Method `inputData_3015()` mengisi data awal, `tampilData_3015()` menampilkan isi playlist, dan `quickSort_3015()` bersama `partition_3015()` menjalankan proses pengurutan.

c. Output

```
<terminated> Sorting_2511533015 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.j
=== Sorting Playlist NIM: 2511533015 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
Pilihan: 2
Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Bella Ciao - 214 detik
5. Volare - 205 detik
6. O Sole Mio - 242 detik
7. Con Te Partiro - 240 detik
Data Setelah Quick Sort (Durasi Asc):
1. La Perla - 196 detik
2. Volare - 205 detik
3. Bella Ciao - 214 detik
4. Con Te Partiro - 240 detik
5. O Sole Mio - 242 detik
6. La Rumba Del Perdon - 252 detik
7. Mio Cristo Piange Diamanti - 270 detik
```